

ICLR 2024 | Why Nonlinear RNNs Struggle to Remember, and How to Loosen the Limit

An inverse approximation theorem shows that any target a nonlinear RNN can stably learn must have exponentially fading memory, and a principled stable reparameterization relaxes the limit. Work by Microsoft Research Asia and the National University of Singapore.

Recurrent neural networks are one of the most basic tools we reach for when data arrives as a sequence: time series, speech, text, sentiment. They are also famous for one recurring disappointment. When the signal depends on something that happened far in the past, plain RNNs tend to lose the thread. The usual explanation is an optimization story: gradients that vanish or explode as they are propagated back through many time steps, making long-range dependencies hard to train.

But that explanation leaves a sharper question unanswered. Is the trouble with long-term memory merely a training artifact that a better optimizer might fix, or is it a structural limit of what an RNN can represent at all? The two diagnoses call for very different responses, and telling them apart requires an approximation-theoretic lens rather than an optimization one.

This ICLR 2024 paper settles the question for nonlinear RNNs. It proves an inverse, or Bernstein-type, approximation theorem: if a nonlinear RNN can stably approximate a target sequence relationship, then that target's memory must decay exponentially in time. In other words, the difficulty with long-term dependencies is baked into the RNN hypothesis space, not just the optimizer. The same analysis then points to a cure, a stable reparameterization of the recurrent weights that provably and empirically relaxes the limit.

Authors: Zhong Li (Microsoft Research Asia), Shida Wang and Qianxiao Li (National University of Singapore; Institute for Functional Intelligent Materials).

Paper: <https://arxiv.org/abs/2305.19190>

Two directions in approximation theory

Approximation theory offers two complementary kinds of statement. A forward, or Jackson-type, theorem tells you how well a model can approximate a target that is sufficiently regular. An inverse, or Bernstein-type, theorem runs the other way: it assumes a target can be approximated efficiently and then deduces what regularity the target must have had in the first place. Most approximation results in machine learning are forward theorems, which is why they rarely expose an architecture's true ceiling. Inverse theorems are exactly the tool for that job.

There was already a precedent. Earlier work proved a Bernstein-type result for linear RNNs, showing that any linear target they can efficiently approximate must have exponentially decaying memory, a phenomenon dubbed the curse of memory. The open and pressing question was whether nonlinearity, which greatly enlarges a model's capacity, would break the

course. Intuitively, more expressive models should be able to hold on to more of the past. This paper asks whether that intuition survives contact with a rigorous inverse theorem.

Two tools: a memory function and stable approximation

The analysis rests on two ingredients. The first is a memory function generalized from the linear setting to arbitrary nonlinear functional sequences. It probes the model with Heaviside step inputs and measures how strongly the output at time t still responds to the input, through the derivative of that response. Crucially, this memory function is not just an abstract definition: it can be numerically queried from a trained model, so you can actually plot how a network's memory decays.

The second ingredient is a notion of stable approximation. Gradient-based training only works if small changes to the parameters cause small changes to the model, so the authors define the perturbation error at hidden dimension m as the worst-case output error when the parameters are perturbed within a ball of radius β . A target is stably approximated if that perturbation error stays continuous up to some positive radius, called the stability radius. This is a mild, optimization-friendly requirement, and it is the hinge of the whole argument.

Figure 1 makes the good case concrete. For a target whose memory decays exponentially, the perturbation-error curves for different hidden dimensions all pass through a common point, and to the left of a positive stability radius the limiting curve stays continuous. A stable approximation exists.

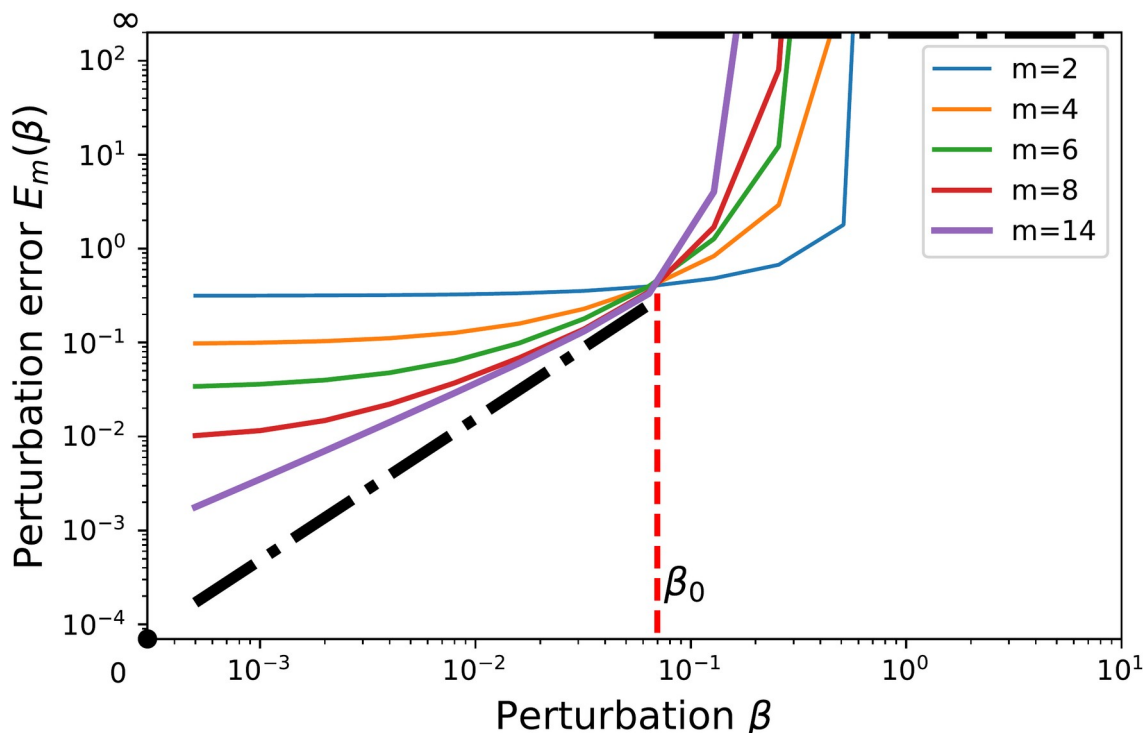


Figure 1. Perturbation error against perturbation strength β for a linear target with exponentially decaying memory, across hidden dimensions $m = 2$ to 14 . A positive stability radius exists (marked in red): to its left the limiting error curve stays continuous, so the target is stably approximable.

The inverse theorem: nonlinearity does not help

With these tools in place, the main theorem is a clean converse statement. If a target with decaying memory is stably approximated by nonlinear RNNs whose weight norms and perturbed memories stay uniformly controlled, then the target's memory must decay exponentially. The proof uses stability to drive the hidden-state derivatives toward zero and then invokes the Hartman-Grobman theorem to bound the eigenvalues of the recurrent matrices, forcing exponential decay. Nonlinearity, it turns out, does not lift the curse of memory; it inherits it.

The numerics tell the same story from the other side. When the target has only polynomially decaying memory, the stability that Figure 1 enjoyed disappears. Instead of converging to a continuous limiting curve, the perturbation-error curves for different hidden dimensions keep intersecting further and further to the left as the dimension grows, so no positive stability radius survives. Figure 2 shows this for a nonlinear tanh RNN, and it matches the linear case exactly.

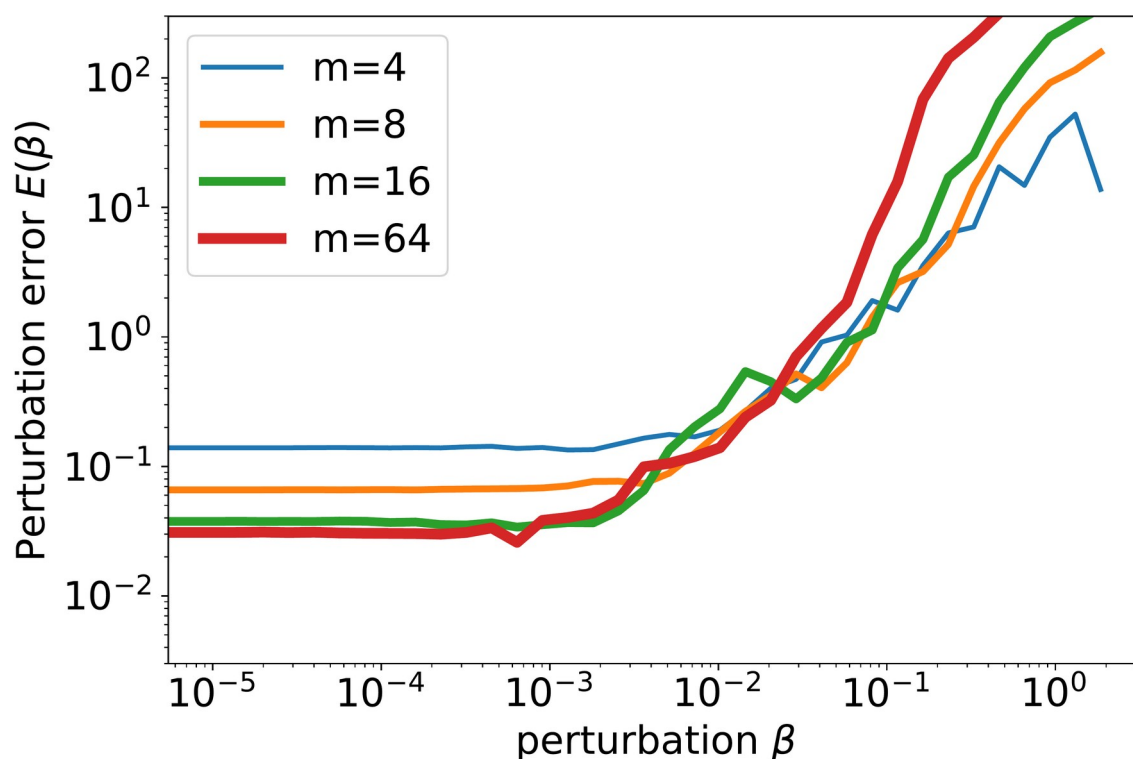


Figure 2. A nonlinear tanh RNN approximating a target with polynomially decaying memory (fit at 1000 epochs), for hidden dimensions $m = 4$ to 64. The curves keep crossing further left as m grows, so there is no positive stability radius: the target cannot be stably approximated.

The most direct evidence comes from a filtering experiment. The authors build several randomly initialized RNNs as teacher models with a large hidden dimension of $m = 256$, then approximate each with a series of student RNNs and numerically test whether the approximation is stable. Keeping only the teachers that are both approximable and stable leaves exactly one kind of survivor: teachers whose memory function decays exponentially. Figure 3 shows the stability test on the left and one surviving teacher's exponentially decaying memory on the right.

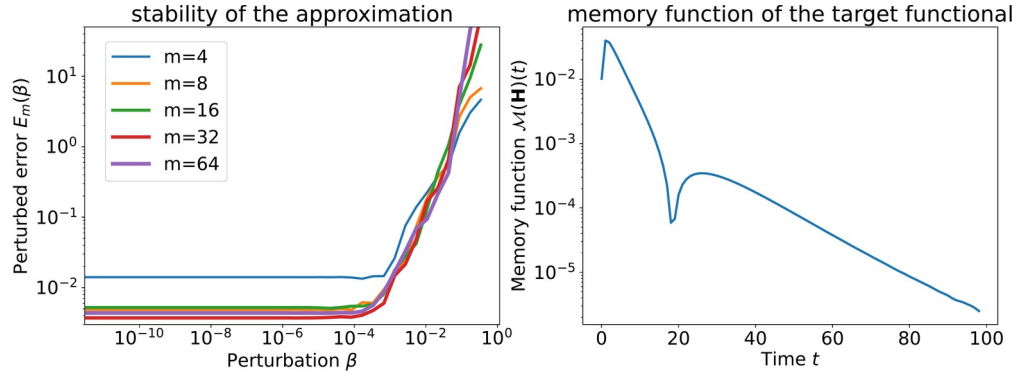


Figure 3. Filtering random teacher RNNs (hidden dimension $m = 256$) by stability. Left: the perturbation error of student approximations across $m = 4$ to 64. Right: the memory function of a surviving teacher, which decays exponentially. Only exponential-memory teachers pass the stability filter.

The cure: stable reparameterization

If the problem is that ordinary training pushes the recurrent weight into regions where small perturbations blow up, the fix is to reparameterize so that every reachable weight is already stable. The authors replace the recurrent weight matrix with a continuous map that always lands in the set of matrices with negative real-part eigenvalues, for example an exponential map or a softplus map. The network can still push its eigenvalues toward the edge of stability to capture more memory, but it can no longer cross the edge, so the approximation stays continuous under perturbation.

Figure 4 confirms the recovery on the synthetic side. Applying exponential and softplus reparameterizations to a linear RNN approximating a polynomial-decay target restores the continuous limiting curve that was missing in Figure 2, exactly what the theory predicts a stable parameterization should do.

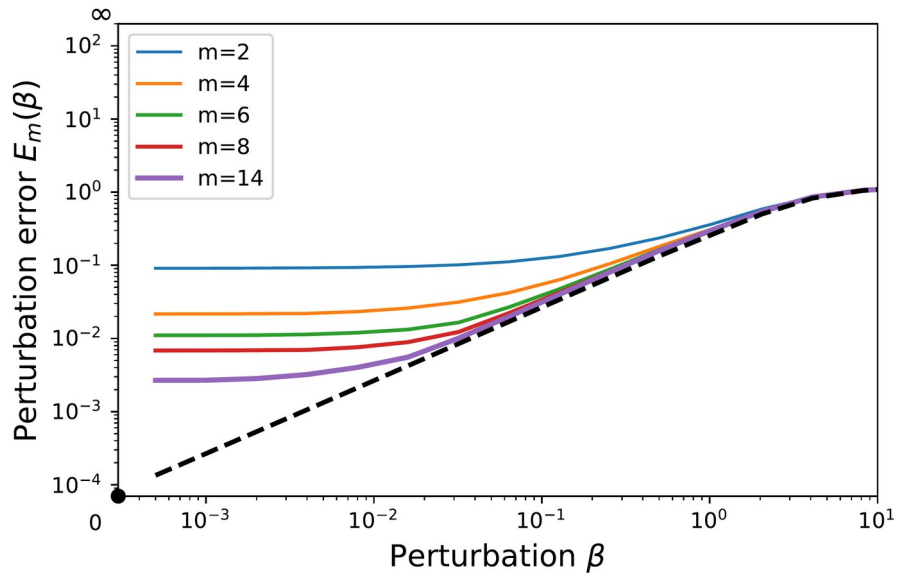


Figure 4. The same polynomial-decay target as before, now approximated by a linear RNN with exponential and softplus reparameterization, across $m = 2$ to 14. The limiting curve is continuous again: stability, and with it approximability, is restored.

Does it help in practice? MNIST

The payoff is not only theoretical. On MNIST image classification, trained for 10 epochs from identical weight initializations and averaged over 3 repeats, the three stable reparameterizations all beat the direct, unstable parameterization that uses the recurrent weight as-is. Softplus reaches the best test accuracy, and because reparameterization does not change the model's inherent capacity, the improvement is an optimization and stability effect rather than a capacity change.

Table 1. Test loss and accuracy of different recurrent-weight parameterizations on MNIST (10 epochs, averaged over 3 repeats). The stable maps all outperform the direct, unstable baseline.

Parameterization	Test loss	Test accuracy (%)
softplus (stable)	0.8903	71.36
exp (stable)	0.9028	70.55
inverse (stable)	0.9082	70.77
direct (unstable)	0.9177	68.47

The gap is modest but consistent: softplus lands at 71.36% against 68.47% for the unstable baseline, a gain of nearly 3 points, with exp and inverse also above 70%. The synthetic experiments sharpen the same point about difficulty. Exponential-memory targets are fit in roughly 10 epochs, whereas polynomial-memory targets need on the order of 1000 epochs and still fail the stability test.

Takeaway

The one-line version is this. No matter how you train them, plain nonlinear RNNs can only stably approximate sequence relationships whose memory fades exponentially, so their well-known struggle with long-term dependencies is a property of the architecture, not merely of the optimizer. That is the honest boundary of the negative result. The constructive half is that the same analysis names the cure: reparameterizing the recurrent weights with a stable map such as exponential or softplus keeps the eigenvalues near the edge of stability without crossing it, provably relaxing the curse of memory and, on a real task like MNIST, training faster and generalizing a little better.

Beyond RNNs, the two tools introduced here, a numerically queryable memory function and a perturbation-based notion of stable approximation, are general enough to extend toward gated architectures like GRU and LSTM. That gives practitioners a principled way to reason in advance about which sequence relationships a given recurrent model can and cannot hope to learn.